

LAB3: Computer Vision

Edge Detection using Sobel and Canny Operators

Objective:

- **Understand the concept of edge detection in images**
- **Study the working of Sobel and Canny edge detectors**
- **Compare traditional gradient-based edge detection with an advanced multi-stage method**
- **Analyze the quality of detected edges under different conditions.**

LAB: Computer Vision

Description

Digital images contain important structural information in the form of edges.

Edges represent sudden changes in intensity, which usually correspond to object boundaries.

Edge detection is a fundamental step in computer vision tasks such as:

- object detection
- Segmentation
- feature extraction
- shape recognition

Different edge detectors produce different results depending on:
noise level threshold values image quality illumination conditions.

LAB: Computer Vision

Object Detection

Finding where an object is in an image and what it is.

LAB: Computer Vision

Segmentation

Dividing an image into meaningful regions or objects.

LAB: Computer Vision

Feature Extraction

Finding important points or patterns in an image..

LAB: Computer Vision

Shape recognition

Identifying objects based on their shape or outline

.

Noise Level

- More noise → more false edges
 - Less noise → cleaner edges
-

◆ Threshold Values

- Low threshold → many edges (including false ones)
 - High threshold → fewer edges (may miss real edges)
-

◆ Image Quality

- Blurry image → weak edges
 - Sharp image → strong edges
-

◆ Illumination Conditions

- Poor lighting → missing edges
- Good lighting → clear boundaries

```
[2]: import cv2
import matplotlib.pyplot as plt

img = cv2.imread("camera.png")
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.imshow(img_rgb)
plt.axis("off")
```

```
[2]: (np.float64(-0.5), np.float64(511.5), np.float64(511.5), np.float64(-0.5))
```



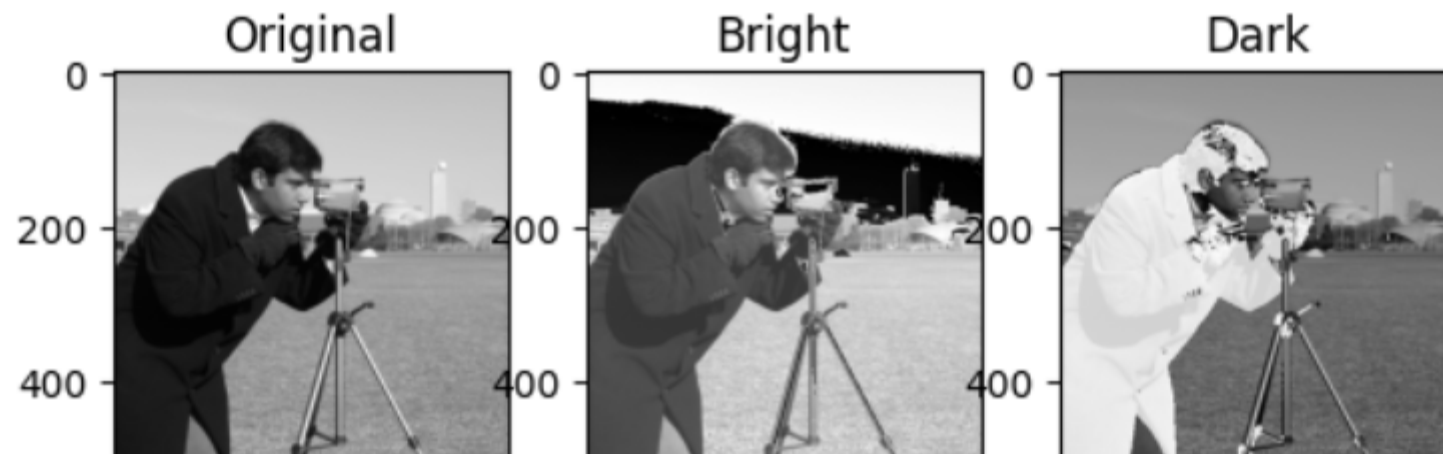
```
[3]: import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("camera.png", 0)

bright = img + 50
dark = img - 50

plt.subplot(1,3,1); plt.title("Original"); plt.imshow(img, cmap='gray')
plt.subplot(1,3,2); plt.title("Bright"); plt.imshow(bright, cmap='gray')
plt.subplot(1,3,3); plt.title("Dark"); plt.imshow(dark, cmap='gray')
```

[3]: <matplotlib.image.AxesImage at 0x1df0f8a9810>



```
[4]: import numpy as np
import matplotlib.pyplot as plt
```

```
X = np.array([1, 2, 3, 4])
Z = np.array([2, 2, 2, 2])
f = 1
```

```
x = f * X / Z
```

```
plt.scatter(X, Z)
plt.title("3D Points")
plt.show()
```

```
plt.scatter(x, np.zeros_like(x))
plt.title("Projected 2D Points")
plt.show()
```

```
[5]: K = np.array([
    [800, 0, 320],
    [0, 800, 240],
    [0, 0, 1]
])
```

```
print(K)
```

```
[[800  0 320]
 [  0 800 240]
 [  0  0  1]]
```

```
[6]: import cv2
import matplotlib.pyplot as plt

img = cv2.imread("camera.png", 0)

edges = cv2.Canny(img, 100, 200)

plt.imshow(edges, cmap='gray')
plt.title("Canny Edge Detection")
plt.axis("off")
```

```
[6]: (np.float64(-0.5), np.float64(511.5), np.float64(511.5), np.float64(-0.5))
```

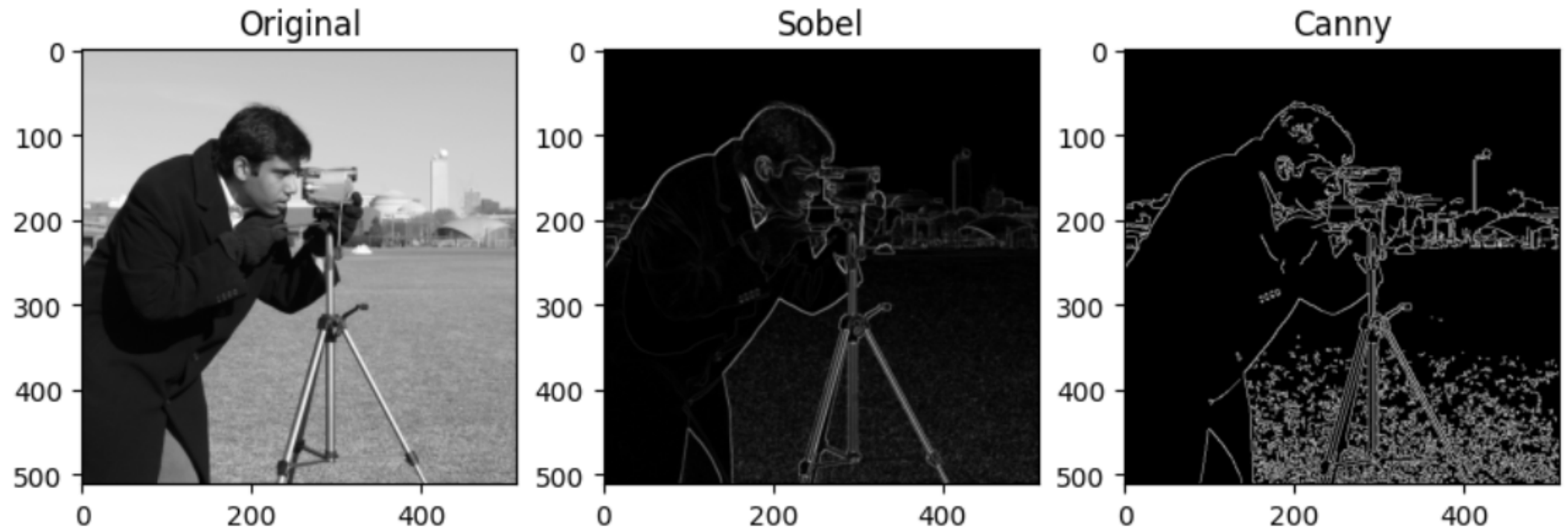
Canny Edge Detection



```
[7]: sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0)
sobely = cv2.Sobel(img, cv2.CV_64F, 0, 1)
sobel = abs(sobelx) + abs(sobely)

canny = cv2.Canny(img, 100, 200)

plt.figure(figsize=(10,4))
plt.subplot(1,3,1); plt.title("Original"); plt.imshow(img, cmap='gray')
plt.subplot(1,3,2); plt.title("Sobel"); plt.imshow(sobel, cmap='gray')
plt.subplot(1,3,3); plt.title("Canny"); plt.imshow(canny, cmap='gray')
plt.show()
```



-Students task:

- Change thresholds
- Observe weak vs strong edges
- Apply to noisy images
- Compare Sobel vs Canny
- Explain which is better and why

- *At the end create pdf file and submit, don't be late, late submission may affect your sessional marks*